Experiment 5

Student Name: Bhuvan Dhiman **UID: 23BCS14067**
Branch: BE-CSE **Section/Group: Krg-3A**
Semester: 5th **Subject Name: ADBMS**
Subject Code: 23CSP-333 **Date: 24 / 09 / 2025**

Aim:

Views: Performance Benchmarking : Normal View vs. Materialized View (Medium)

Problem A:
1. Create a large dataset:

- Create a table names transaction_data (id , value) with 1 million records.
    - take id 1 and 2, and for each id, generate 1 million records in value column
- Use Generate_series () and random() to populate the data.

2. Create a normal view and materialized view to for sales_summary, which includes total_quantity_sold, total_sales, and total_orders with aggregation.

3. Compare the performance and execution time of both.

Views: Securing Data Access with Views and Role-Based Permissions (Hard)

The company **TechMart Solutions** stores all sales transactions in a central database. A new reporting team has been formed to analyze sales but **they should not have direct access to the base tables** for security reasons.
The database administrator has decided to:

1. Create **restricted views** to display only summarized, non-sensitive data.

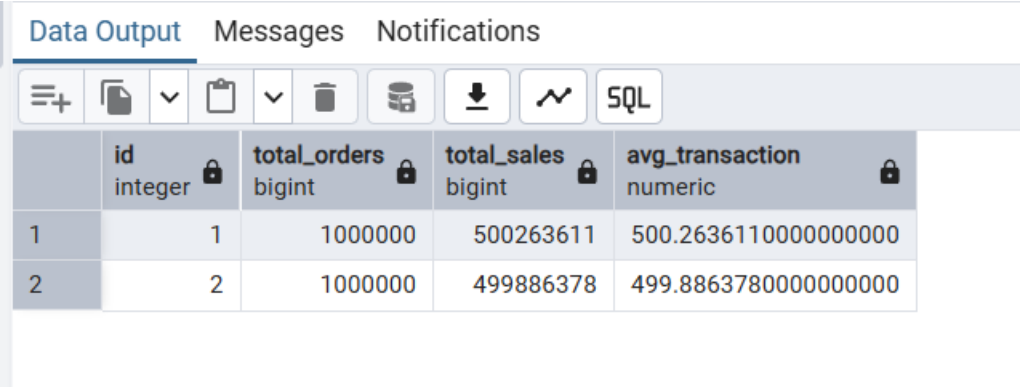
2. Assign access to these views to specific users using **DCL commands** (GRANT, REVOKE).

Code:

Problem A:

```
CREATE TABLE transaction_data (
   id INT,
   value INT
);

INSERT INTO transaction_data (id, value)
SELECT 1, random() * 1000
FROM generate_series(1,1000000);

INSERT INTO transaction_data (id, value)
SELECT 2, random() * 1000
FROM generate_series(1,1000000);

CREATE OR REPLACE VIEW sales_summary_view AS
SELECT
   id,
   COUNT(*) AS total_orders,
   SUM(value) AS total_sales,
   AVG(value) AS avg_transaction
FROM transaction_data
GROUP BY id;

EXPLAIN ANALYZE
SELECT * FROM sales_summary_view;

DROP MATERIALIZED VIEW IF EXISTS sales_summary_mv;

CREATE MATERIALIZED VIEW sales_summary_mv AS
SELECT
   id,
   COUNT(*) AS total_orders,
   SUM(value) AS total_sales,
   AVG(value) AS avg_transaction
FROM transaction_data
GROUP BY id;


EXPLAIN ANALYZE
SELECT * FROM sales_summary_mv;

REFRESH MATERIALIZED VIEW sales_summary_mv;
```

```
SELECT * FROM transaction_data LIMIT 10;
SELECT * FROM sales_summary_view;
SELECT * FROM sales_summary_mv;
```



Data Output | Messages | Notifications

	id integer	total_orders bigint	total_sales bigint	avg_transaction numeric
1	1	1000000	500263611	500.2636110000000000
2	2	1000000	499886378	499.8863780000000000

Problem B:

```
CREATE TABLE customer_master (
   customer_id VARCHAR(5) PRIMARY KEY,
   full_name VARCHAR(50),
   phone VARCHAR(15),
   email VARCHAR(50),
   city VARCHAR(30)
);

CREATE TABLE product_catalog (
   product_id VARCHAR(5) PRIMARY KEY,
   product_name VARCHAR(50),
   unit_price NUMERIC(10,2)
);

CREATE TABLE sales_orders (
   order_id SERIAL PRIMARY KEY,
   product_id VARCHAR(5) REFERENCES product_catalog(product_id),
   customer_id VARCHAR(5) REFERENCES customer_master(customer_id),
   quantity INT,
   discount_percent NUMERIC(5,2),
   order_date DATE
);

-- -------- Step 2: Insert Sample Data --------
INSERT INTO customer_master VALUES
('C1', 'Amit Sharma', '9876543210', 'amit.sharma@example.com', 'Delhi'),
('C2', 'Priya Verma', '9876501234', 'priya.verma@example.com', 'Mumbai'),
('C3', 'Ravi Kumar', '9988776655', 'ravi.kumar@example.com', 'Bangalore');
```

```
INSERT INTO product_catalog VALUES
('P1', 'Smartphone X100', 25000),
('P2', 'Laptop Pro 15', 65000),
('P3', 'Wireless Earbuds', 5000);

INSERT INTO sales_orders (product_id, customer_id, quantity, discount_percent, order_date) VALUES
('P1','C1',2,5,'2025-09-01'),
('P2','C2',1,10,'2025-09-02'),
('P3','C3',3,0,'2025-09-03');

-- -------- Step 3: Create Restricted View --------
CREATE OR REPLACE VIEW vw_sales_summary AS
SELECT
   P.product_id,
   P.product_name,
   SUM(O.quantity) AS total_units_sold,
   SUM(O.quantity * P.unit_price * (1 - O.discount_percent/100)) AS total_revenue
FROM sales_orders O
JOIN product_catalog P ON O.product_id = P.product_id
GROUP BY P.product_id, P.product_name;

CREATE ROLE reporting_team WITH LOGIN PASSWORD 'report123';

GRANT CONNECT ON DATABASE test TO reporting_team;
GRANT USAGE ON SCHEMA public TO reporting_team;

GRANT SELECT ON vw_sales_summary TO reporting_team;

REVOKE SELECT ON sales_orders FROM reporting_team;
REVOKE SELECT ON product_catalog FROM reporting_team;
REVOKE SELECT ON customer_master FROM reporting_team;


SELECT * FROM vw_sales_summary;
```

Data Output | Messages | Notifications

Showing rov

	product_id character varying (5)	product_name character varying (50)	total_units_sold bigint	total_revenue numeric
1	P3	Wireless Earbuds	3	15000.000000000000000000000000
2	P2	Laptop Pro 15	1	58500.000000000000000000000000
3	P1	Smartphone X100	2	47500.000000000000000000000000